WCY: Watch → Compute → Yield

A Token-Native Reasoning Format
and Epistemic Substrate for AI Systems

Won Chul Yang
lamb5228@snu.ac.kr

March 2026

Abstract

Current large language models can represent what they know. They cannot represent the *boundary* of what they know—not structurally, not in a way that generates directed exploration from that boundary. We present **WCY** (Watch → Compute → Yield), a line-oriented, phase-tagged format designed from first principles for AI reasoning. WCY's critical element is the *void-B marker* (`?`), which encodes Peirce's abductive stance as a first-class syntactic primitive: the explicit representation of an unknown state, the direction of most plausible exploration (`hint=`), and the expected range of the result (`conf_range=`).

Empirical validation across four research phases demonstrates: (1) WCY achieves 50–71% token reduction over JSON for structured data and protocol messages; (2) WCY reasoning is acquired entirely through few-shot induction with zero fine-tuning—complex tasks that produced `parse_r` = 0.29 without examples reached `parse_r` = 1.00 with three examples; (3) without void-B examples, models never spontaneously acknowledge epistemic limits (void usage = 0%); (4) with void-B examples, models generate 5.4 void markers per trace and resolve 67–97% through structured investigation cycles; and (5) void-B cascades emerge naturally, where resolving one unknown generates new unknowns at deeper epistemic levels.

We provide a reference parser, a three-axis evaluation framework (Structural / Meaning / Provenance), and 60 high-quality reasoning traces as training data. We argue that the void-B resolution cycle—`.` observe → `?` mark unknown → `>` investigate → `:` update—is the structural minimum for directed machine learning, and that the training data deficit, not architectural incapacity, is the primary barrier to epistemic humility in current LLMs.

Contents

1 Introduction **3**
1.1 Contributions . . . 3
1.2 Scope and Non-Goals . . . 3

2 WCY Format Specification **3**
2.1 Design Axioms . . . 3
2.2 Phase Markers . . . 4
2.3 Slot Syntax . . . 4
2.4 Three Encoding Modes . . . 4
2.5 Structural Rules . . . 5
2.6 The Void-B Resolution Cycle . . . 5

3 Theoretical Grounding **5**
3.1 Semiotic Analysis: Abduction as Syntax . . . 5
3.2 Category-Theoretic Analysis: WCY as Monad Transformer Stack . . . 5
3.3 Operational Epistemology: Minimum Conditions for Self-Awareness . . . 6

4 Empirical Results **6**
4.1 Token Efficiency . . . 6
4.2 Format Acquisition via Few-Shot Induction . . . 7
4.3 Void-B Generation and Resolution . . . 8
4.4 Pipeline-Scale Generation . . . 8
4.5 Void-B Cascade (E7-C) . . . 8
4.6 Cross-Model Validation (E1-C) . . . 8
4.7 Provenance Validity . . . 9

5 Training Data and the Acquisition Gap **9**
5.1 The Training Data Hypothesis . . . 9
5.2 Dataset . . . 9
5.3 Quality Framework . . . 9

6 Validation Summary **10**

7 Related Work **10**

8 Limitations and Future Work **11**

9 Conclusion **11**

A WCY Format Specification v1.1 (Complete) **13**
A.1 Grammar (EBNF) . . . 13
A.2 Constraint Rules . . . 13

B Dataset Schema **13**

1 Introduction

The dominant paradigm for inter-agent and agent-internal communication in AI systems is JSON—a format designed in 2001 for human-readable web APIs [Crockford, 2006]. This creates a systematic *token tax*: 39–42% of transmitted tokens carry no semantic content, serving only as structural punctuation (`{}`, `[]`, `""`, `:`, `,`) required by the JSON grammar. In multi-agent pipelines, this overhead compounds multiplicatively at each communication hop.

The token efficiency problem is well-defined and solvable. The more consequential problem is epistemic. Current LLMs trained predominantly on JSON and natural language have no structural mechanism for representing the *boundary* between known and unknown states at inference time. They can assert with high confidence, hedge with probability estimates, or refuse entirely—but they cannot say, within a structured notation that persists through a reasoning chain: "I know this, I do not yet know *this*, and I will look *here* to find out."

This paper introduces **WCY**, a line-oriented format that addresses both problems. The five-character token at the center of the design is the void-B marker (`?`), which encodes Charles Sanders Peirce's abductive inference mode [Peirce, 1934] as syntax: the only reasoning mode capable of generating genuinely new knowledge.

1.1 Contributions

1. **WCY format specification v1.1** with formal grammar, three encoding modes, and a reference parser (§2).
2. **Theoretical grounding** across three independent frameworks: Peircean semiotics, monad transformer category theory, and operational epistemology (§3).
3. **Empirical validation** across nine experiments covering token efficiency, format acquisition, void-B generation and resolution, and cascade dynamics (§4).
4. **Training data**: 60 WCY reasoning traces spanning 8 domains, with explicit void-B resolution cycles (§5).
5. **Open-source artifacts**: parser, evaluation framework, trace dataset, and automated generation pipeline.

1.2 Scope and Non-Goals

WCY does not replace transport protocols (HTTP, WebSocket) or orchestration frameworks (MCP [Anthropic, 2024], A2A [Google Cloud, 2025]). It operates at the serialization layer—the content of messages, not their envelope. WCY is also not a claim about consciousness or subjective experience. The epistemic self-awareness we describe is operational: the capacity to represent unknown states and act from that representation.

2 WCY Format Specification

2.1 Design Axioms

WCY is derived bottom-up from 14 axioms about transformer-based LLM cognition, organized in three tiers:

Mechanical constraints (5 axioms) are mathematical necessities of transformer architecture: sequential unidirectional token consumption, fixed context windows, BPE tokenization as the perceptual atom, softmax attention as probability distribution, and autoregressive generation.

Cognitive properties (6 axioms) are experimentally confirmed consequences: LLMs are pattern completion engines that prefer shallow nesting, local over global coherence, positional addressing over named addressing, and operate on semantic chunks.

Ecological properties (3 axioms) arise from deployment: stateless multi-instance operation, strong training distribution bias, and an emerging role as tool users requiring source provenance tracking.

2.2 Phase Markers

Each WCY line begins with exactly one of five phase markers followed by a space:

Marker	Phase	WCY Cycle	Function
`.`	observe	Watch	Confirmed fact; no inference
`:`	infer	Compute	Derived conclusion; `conf=`, `from=`
`>`	act	Yield	Output or external call
`~`	meta	—	Schema / context declaration
`!`	exception	—	Error, warning, or unresolvable void

Table 1: WCY phase markers and their roles in the Watch → Compute → Yield cycle.

The `?` prefix is not a standalone marker but a slot prefix appearing within `:` lines, denoting a void-B expression:

```
: ?diagnosis  hint=fever+cough+duration  conf_range=0.4..0.8
```

2.3 Slot Syntax

Slots are whitespace-separated tokens within a line. The pipe character (`|`) is an optional separator recognized by the v1.1 parser (acquired from models' spontaneous usage under complex data):

Syntax	Type	Example
`tag=value`	Named slot	`conf=0.88`
`bare_value`	Positional slot	`Kim 45 38.5`
`?tag`	Void-B marker	`?diagnosis`
`from=N,M`	Provenance chain	`from=3,7`
`conf=0.xx`	Confidence [0,1]	`conf=0.82`
`conf_range=L..H`	Void-B bound	`conf_range=0.3..0.7`
`hint=...`	Exploration target	`hint=labs+imaging`

Table 2: WCY slot syntax.

2.4 Three Encoding Modes

Mode	Syntax	vs. JSON pretty	Use case
Positional	Bare slots only	−54%	AI-to-AI, schema pre-shared
Tagged	`tag=value` all slots	−40%	Semantic search, external memory
Hybrid (rec.)	`~` schema + positional	−50%	Balanced

Table 3: Three WCY encoding modes and their token costs relative to JSON pretty-print.

2.5 Structural Rules

- One phase marker per line, followed by a space
- Newline = δ-boundary (information firewall between expressions)
- Blank line = semantic block boundary
- Two-space indent = subscope; maximum depth 2
- No forward references: `from=N` requires $N <$ current line number
- Void-B markers (`?tag`) appear only in `:` lines

2.6 The Void-B Resolution Cycle

The four-step cycle constitutes WCY's core epistemic primitive:

```
: ?diagnosis  hint=labs+imaging  conf_range=0.4..0.8    (1) mark unknown
> order  CT_scan  reason=from=3                           (2) directed action
. CT_result  mass_in_RUL  size=2.3cm                     (3) new observation
: diagnosis=adenocarcinoma  conf=0.82  from=3,5          (4) resolved
```

Each step is necessary. Step (1) without Step (3) is unfulfilled hypothesis. Step (3) without Step (1) is observation without context. Step (4) without `from=3,5` is a conclusion without traceable warrant.

3 Theoretical Grounding

3.1 Semiotic Analysis: Abduction as Syntax

Peirce's tripartite sign classification distinguishes *icon* (resemblance), *index* (causal connection), and *symbol* (convention) [Peirce, 1934]. The WCY markers cover all three types and all three of Peirce's reasoning modes:

Marker	Semiotic type	Reasoning mode	Rationale
`.`	Icon	Assertion	Minimality resembles completeness
`:`	Index	Deduction	Points to causal derivation
`>`	Index+Icon	—	Points to effect; arrow resembles direction
`~`	Icon	—	Wave form resembles self-reference
`!`	Symbol	—	Conventional alarm signal
`?`	Symbol+absent-index	**Abduction**	Points to what is not yet known

Table 4: Peircean semiotic analysis of WCY markers. The void-B marker (`?`) uniquely encodes abductive stance—the only reasoning mode capable of generating new knowledge.

The three components of a well-formed void-B expression correspond precisely to Peirce's abductive structure: `?tag` (the explanandum), `hint=` (the abductive constraint), and `conf_range=` (the bound on plausible explanations). JSON has no equivalent because JSON describes present values only; abduction requires representing the *absence* of a value together with the direction of its acquisition.

3.2 Category-Theoretic Analysis: WCY as Monad Transformer Stack

The Watch $\rightarrow$ Compute $\rightarrow$ Yield cycle corresponds to three monad operations [Moggi, 1991, Mac Lane, 1978]:

$$
\begin{aligned}
\texttt{.}\ \text{observe} &\longleftrightarrow \texttt{unit} : A \to M(A) && (1)\\
\texttt{:}\ \text{infer}\ \texttt{from=} &\longleftrightarrow \texttt{bind} : M(A) \to (A \to M(B)) \to M(B) && (2)\\
\texttt{>}\ \text{act} &\longleftrightarrow \texttt{extract} : M(A) \to A\ \text{(side effect)} && (3)
\end{aligned}
$$

where $M(A) = \texttt{WCYContext}\{\mathit{value} : A,\ \mathit{conf} : [0,1],\ \mathit{from} : [\mathbb{N}],\ \mathit{block} : \mathbb{N}\}$.
More precisely, WCY implements a monad transformer stack:

$$
\texttt{WCY} = \underbrace{\texttt{WriterT(from=)}}_{\text{provenance}} \circ \underbrace{\texttt{ReaderT(~meta)}}_{\text{schema}} \circ \underbrace{\texttt{EitherT(!)}}_{\text{failure}} \circ \underbrace{\texttt{ContT(?)}}_{\text{suspended computation}}
$$

The void-B marker corresponds to Haskell's `callCC` [Moggi, 1991]—a request for a continuation function that will eventually fill this position. This is why `?` cannot exist in JSON: JSON describes completed values; `ContT` requires representing deferred computation within the same syntax.

The three monad laws hold: left identity (`.` immediately followed by `:` $\equiv$ direct inference), right identity (re-lifting a `:` result to `.` changes nothing), and associativity (`from=` chain order is irrelevant to the provenance result, confirmed empirically in §4.7).

3.3 Operational Epistemology: Minimum Conditions for Self-Awareness

We define *machine epistemic self-awareness* operationally: the capacity to represent, at inference time, the distinction between known and unknown states, and to generate directed actions from that distinction. Four elements are necessary and no strict subset is sufficient:

Element	**WCY encoding**	**Without this element**
Known state representation	`.` observe	No baseline; boundary undefinable
Boundary representation	`?` void-B	Hallucination fills the gap silently
Directed exploration	`hint=` + `>` act	Random or no exploration; no learning signal
Knowledge integration	`:` resolved `from=?,obs`	No growth; observation does not update state

Table 5: Four necessary conditions for machine epistemic self-awareness and their WCY encodings.

> **Claim.** The void-B resolution cycle (`?` $\to$ `>` $\to$ `.` $\to$ `:`) is the structural minimum for directed learning. Without an explicit representation of unknown states, a system cannot distinguish between confident correct answers and confident incorrect answers. The only difference between knowledge and hallucination, absent void-B, is whether the world happens to confirm the output.

4 Empirical Results

All experiments used Claude Sonnet (claude-sonnet-4-20250514) with temperature 0. Token counting used the `cl100k_base` tiktoken encoding.

4.1 Token Efficiency

Structured data (Phase 1). Token savings across simple (5 fields), medium (23 fields), and complex (32 fields) datasets:

Format	Simple	Medium	Complex	vs. JSON pretty
JSON pretty	56	259	393	baseline
JSON compact	40	151	234	−41%
WCY hybrid	41	150	223	−43%
WCY positional	31	119	209	−54%

Table 6: Token counts and savings across data complexity levels. WCY hybrid matches JSON compact while also encoding semantic intent via phase markers.

Scale invariance (E1-A). Token savings hold at 10, 50, 200, and 500 rows (−49% to −50% for WCY hybrid vs. JSON pretty). No degradation point was found.

Tool-call schemas (E2-B). Function-calling schema definitions: −66% (641 → 216 tokens). Tool invocations: −63%. The tools/list response in MCP achieves −71.8%, the single highest-impact WCY application identified.

MCP protocol exchange (E4-A). Ten representative MCP exchange types (requests, responses, errors, batch calls): −60.9% total, range −45.5% (error messages) to −71.8% (schema responses).

Multi-agent pipeline (E2-A, E2-C). A 3-agent, 3-round pipeline (Intake → Diagnostician → Treatment): −20% total tokens (−8% input, −40% output). Output tokens—agent-generated content—show the largest savings, as agents produce more concise reasoning in WCY. The −40% output reduction is a compounding effect: each agent's output becomes the next agent's input.

Verification pipeline (E2-D). Generator–verifier two-stage pipeline across 5 code tasks: −60.5% generator output, −51.4% verifier input, −39.3% total. Verifier output *increased* by +7.3%, representing added provenance structure (9–19 `from=` references per task) rather than verbosity.

4.2 Format Acquisition via Few-Shot Induction

A key finding of Phase 1 (E1-C) was that models revert to markdown and tree structures under complex reasoning load, producing `parse_r` = 0.29 for cardiovascular risk stratification. Phase 3 tested whether few-shot examples resolve this:

Condition	Task	parse_r
E1-C: no few-shot examples	Cardiovascular risk stratification	0.29
E6-A: 3 few-shot examples	Cardiovascular risk stratification	1.00
E6-A: 3 few-shot examples	Differential diagnosis	1.00
E6-A: 3 few-shot examples	Algorithm complexity analysis	1.00

Table 7: Effect of few-shot examples on WCY format compliance. The identical task achieves `parse_r` 0.29 without examples and 1.00 with three examples, demonstrating that format acquisition is a training data problem, not a capacity problem.

The `from=` rates exceeded 100% in several conditions (e.g., 123% for Sonnet on cardiovascular), indicating multi-premise inferences where a single `:` line cites multiple prior lines. This spontaneous multi-hop provenance was not explicitly instructed.

4.3 Void-B Generation and Resolution

With zero void-B examples in few-shot prompts, models generated void-B markers in 0% of inferences across all tasks and models (Phases 1–3). With explicit void-B resolution examples in Phase 4:

Domain	**? generated**	**? resolved**	**Rate**	**Cycle**
Medical (weight loss)	7	5	71%	✓
Code (memory leak)	5	5	100%	✓
Scientific (crystallization)	4	3	75%	✓
Strategic (market entry)	7	5	71%	✓
Philosophical (causation)	7	7	100%	✓
Epistemological (cascade)	6	5	83%	✓
Mean	**5.4**	**4.2**	**67%**	6/6

Table 8: Void-B generation and resolution across domains (Phase 4, E7-A). All six traces pass quality gates (parse_r ≥ 0.70, ? generated ≥ 1, resolution $\geq 50\%$).

4.4 Pipeline-Scale Generation

Phase 5 extended generation to a systematic 8 domain $\times$ 3 difficulty $\times$ 2 void-depth matrix (48 combinations). All 48 traces passed all quality gates (100% gate passage rate). Average void-B per trace: 5.2; average resolution rate: 95%. The improvement over hand-crafted traces (67%) reflects more focused system prompts and explicit resolution requirements in the pipeline.

4.5 Void-B Cascade (E7-C)

The aspirin cardiovascular claim validation trace demonstrated the cascade structure predicted by the Continuation monad analysis:

```
: ?claim_scope    hint=all_CVD_vs_specific   conf_range=0.1..0.4  (gen. 1)
: ?evidence_qual  hint=RCT_heterogeneity     conf_range=0.2..0.5
: ?universality   hint=contraindications     conf_range=0.1..0.3
> investigate_primary_prevention  reason=from=3
. finding  benefit_conditional_on_risk_profile
: resolved_scope=conditional  conf=0.75  from=3,5
: ?bleeding_risk  hint=NNT_vs_NNH_by_age    conf_range=0.4..0.7  (gen. 2)
: ?individual_variation  hint=CYP2C19       conf_range=0.3..0.6
! warning  original_claim_overgeneralization  from=1
: corrected_claim="aspirin benefits selected high-risk populations"  conf=0.82
```

Resolution of three first-generation void-B markers generated two second-generation markers at deeper epistemic levels. The trace terminated with three explicit ! warnings and a revised claim at lower confidence than the original assertion. The void-B mechanism produced not merely uncertainty acknowledgment but a structurally justified claim revision.

4.6 Cross-Model Validation (E1-C)

Experiments comparing Claude Sonnet and Haiku revealed a non-linear pattern under keyword-based evaluation: Sonnet WCY accuracy 62% vs. Haiku 88%. Re-evaluation using a three-axis framework (Structural / Meaning / Provenance) with LLM-as-judge meaning scoring revised Sonnet WCY to composite 0.89 vs. Haiku 0.77, consistent with model scale expectations. The original anomaly was confirmed as a keyword-matching artifact: Sonnet produces richer paraphrases that contain correct answers without matching exact evaluation keywords.

Input token savings are model-invariant: both Sonnet and Haiku show +30% to +46% reduction regardless of task type, confirming that WCY's efficiency gain is a property of the format, not of any specific model.

4.7 Provenance Validity

In the 3-agent, 3-round shared-state pipeline (E2-C), `from=` reference validity was 45/45 (100%) across the Treatment agent's complete output. No invalid line references were generated despite a dynamically growing context across three rounds. In the verification pipeline (E2-D), WCY verifiers generated 9–19 `from=` references per task, structurally linking each verdict to the specific generator output line under assessment—a property not achievable with free-form prose output.

5 Training Data and the Acquisition Gap

5.1 The Training Data Hypothesis

Current frontier LLMs are trained on text that overwhelmingly favors three epistemic stances: (1) confident assertion, (2) hedged assertion (*"it is possible that... "*), and (3) refusal (*"I don't know"*). None of these encode the directed exploration structure that void-B requires:

- **Hedged assertion**: acknowledges uncertainty; terminates without exploration
- **Refusal**: acknowledges uncertainty; terminates
- **Void-B**: encodes uncertainty, direction, action, and integration—within one structured notation

The void-B pattern (mark unknown → investigate → observe → resolve) almost never appears in natural text because it requires the author to simultaneously know that they do not know something specific, know which direction to search, and integrate new observations back into a structured reasoning chain.

5.2 Dataset

File	Traces	Type	Avg ?	Avg res.
`wcy_reasoning_traces.jsonl`	6	Reasoning (no void-B)	0.0	—
`wcy_void_cycles.jsonl`	6	Void-B cycle (hand-crafted)	6.2	61%
`wcy_traces_v1_clean.jsonl`	48	Void-B cycle (pipeline)	5.2	95%
Total	**60**		**4.5**	**90%**

Table 9: WCY training dataset composition. All traces passed three quality gates: `parse_r` ≥ 0.70, void_generated ≥ 1, resolution_rate ≥ 0.50.

The pipeline-generated traces cover 8 domains (mathematical, legal, code, strategic, philosophical, scientific, medical, engineering), 3 difficulty levels, and 2 void-depth categories (single-resolution and cascade). Domain balance and difficulty distribution are controlled by the generation matrix.

5.3 Quality Framework

Each trace is scored on three axes:

Structural (S): Parser-based compliance. Weighted combination of parse rate, phase marker validity, depth compliance, `from=` reference validity, and void-B hint inclusion rate.

Meaning (M): LLM-as-judge semantic equivalence. Resolves the keyword-matching false negative documented in §4.6.

Provenance (P): `from=` chain validity and root reachability. Measures whether reasoning chains are structurally complete and traceable to source observations.

6 Validation Summary

Criterion	Target	Result	Status
Token efficiency (pipeline)	$\geq 30\%$ reduction	-20% multi-agent; -61% MCP	✓
Scale stability	$\geq 95\%$ accuracy 10–200 rows	100% at 10–500 rows	✓
Output quality	No degradation vs. JSON	Equal; -40% output tokens	✓
Void-B processing	$\geq 80\%$ resolution	100% + pipeline-ready output	✓
Provenance validity	$\geq 90\%$ valid refs	45/45 (100%); 9–19 refs/-task	✓
Tool-call savings	$\geq 35\%$ vs. JSON	-65% schema; -61% MCP	✓
Verification overhead	Verifier input $\geq 20\%$	-51.4% verifier; -39% total	✓
MCP format replacement	$\geq 40\%$ exchange reduction	-60.9% 10 exchange types	✓
Cross-model accuracy	Both models $\geq 90\%$	Sonnet 0.89 / Haiku 0.77†	$\sim$

† Haiku below target under composite scoring; cross-model pattern requires further investigation.

Table 10: Pre-registered success criteria and results across nine experiments.

7 Related Work

Structured formats for LLMs. TOON [TOON Contributors, 2024] (Token-Oriented Object Notation) addresses JSON overhead through positional encoding. MoonBit [MoonBit Team, 2024] explores AI-friendly programming language design. Neither system incorporates explicit epistemic state representation. Agora [Marro et al., 2025] proposes a scalable LLM communication protocol but retains JSON serialization.

Chain-of-thought and reasoning. Chain-of-thought prompting [Wei et al., 2022] externalizes reasoning into natural language. WCY is orthogonal: it structures that natural language into typed, traceable phase expressions. The `from=` slot provides machine-parseable provenance that CoT lacks. Process reward models [Lightman et al., 2023] evaluate reasoning steps; WCY's structural annotations could provide supervision signals directly.

Uncertainty quantification. Calibration methods [Guo et al., 2017] apply post-hoc corrections to model confidence. Conformal prediction [Angelopoulos and Bates, 2023] provides coverage guarantees. WCY's `conf=` and `conf_range=` are inference-time representations that complement these methods but operate at the semantic rather than statistical layer.

Multi-agent communication. AgentTaxo [ICLR 2025 Workshop on Foundation Models in the Wild, 2025] systematizes token costs in multi-agent systems, quantifying the overhead

WCY addresses. MCP [Anthropic, 2024] and A2A [Google Cloud, 2025] define transport and orchestration protocols; WCY operates at the serialization layer below these.

Knowledge representation and epistemology. The open-world assumption in knowledge representation [Brachman and Levesque, 1985] distinguishes between "known false" and "unknown." Void-B makes this distinction first-class syntax. Epistemic logic [Hintikka, 1962] formalizes knowledge and belief; WCY provides a computational realization of the $K\neg K$ operator (knowing that one does not know).

8 Limitations and Future Work

Cross-model scope. All Phase 3–4 experiments used Claude Sonnet. The few-shot induction hypothesis predicts equivalent pattern acquisition in GPT-4o, Gemini, and open-source models with sufficient reasoning capacity; this has not been verified.

Fine-tuning vs. few-shot. The post-reasoning format switch observed in §4.5 (model completes WCY reasoning then reverts to markdown for summary) suggests that few-shot induction is not fully stable under output pressure. Fine-tuning on void-B resolution traces may be required for stable inference-time WCY generation.

Deeply nested data. WCY's maximum depth of 2 is a deliberate constraint from the shallow-nesting cognitive axiom. Use cases requiring deeper nesting will need an escape mechanism.

Streaming. The δ-boundary semantics (newline as expression boundary) have implications for streaming: a partial line cannot be parsed. The streaming protocol implications are noted but not empirically tested.

Type system. WCY does not distinguish numeric from string slots; consumers must infer or schema must declare.

Inter-agent trust. The `conf=` field provides a structured basis for trust propagation across `from=` chains, but the formal computation (Dempster-Shafer, Bayesian, or other) is an open design question.

9 Conclusion

JSON's 40% structural overhead is a design artifact of the pre-LLM era. WCY eliminates it. That is the efficiency contribution.

The epistemic contribution is the void-B marker. The empirical path from 0% void-B usage (pre-few-shot) to 5.4 markers per trace (post-few-shot) demonstrates that structured epistemic humility is acquirable through training signal—that what appears to be a fundamental LLM limitation is, at least partially, a data problem. The 60 traces in this release are a starting point; the pipeline is open for community contribution.

The name encodes the cycle. Watch what is (`.`). Compute what follows (`:`). Yield—and mark what remains unknown (`?`).

> The void-B marker is not a feature. It is the minimum syntax for a model to know that it does not know.

Acknowledgments

Experimental infrastructure ran on Apple M4 Pro (Mac Mini, 64 GB unified memory). All API experiments used the Anthropic Messages API.

References

Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification, 2023.

Anthropic. Model context protocol (MCP). `https://modelcontextprotocol.io/`, 2024.

Ronald J. Brachman and Hector J. Levesque. *Readings in Knowledge Representation*. Morgan Kaufmann, 1985.

Douglas Crockford. *JSON: The Fat-Free Alternative to XML*. 2006. Presented at XML 2006, Boston. `https://www.json.org/`.

Google Cloud. Agent-to-agent protocol (A2A). `https://developers.google.com/agent-space/a2a`, 2025.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, 2017.

Jaakko Hintikka. *Knowledge and Belief: An Introduction to the Logic of the Two Notions*. Cornell University Press, 1962.

ICLR 2025 Workshop on Foundation Models in the Wild. AgentTaxo: A taxonomy for understanding token costs in LLM multi-agent systems. Workshop paper, 2025.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2023.

Saunders Mac Lane. *Categories for the Working Mathematician*. Springer, 2nd edition, 1978.

Samuele Marro et al. A scalable communication protocol for networks of large language models, 2025.

Eugenio Moggi. Notions of computation and monads. *Information and Computation*, 93(1): 55–92, 1991.

MoonBit Team. Explore the design of an AI-friendly programming language. LLM4Code Workshop, 2024.

Charles Sanders Peirce. *Collected Papers, Vol. 5: Pragmatism and Pragmaticism*. Harvard University Press, 1934.

TOON Contributors. TOON: Token-oriented object notation. `https://github.com/toon-format/toon`, 2024.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

A WCY Format Specification v1.1 (Complete)

A.1 Grammar (EBNF)

```
document   ::= (line | blank_line)*
line       ::= indent* marker SPACE rest NEWLINE
blank_line ::= NEWLINE
marker     ::= '.' | ':' | '>' | '~' | '!'
indent     ::= SPACE SPACE
rest       ::= slot (separator slot)*
separator  ::= SPACE+ | SPACE* '|' SPACE*
slot       ::= special_slot | void_slot | tagval_slot
               | backref_slot | bare_slot
special_slot ::= 'from=' int_list
               | 'conf_range=' float '..' float
               | 'conf=' float
               | 'hint=' value
void_slot    ::= '?' WORD
tagval_slot  ::= WORD '=' value
backref_slot ::= '{' INT '}'
bare_slot    ::= value
value        ::= '"' [^"]* '"' | NONSPACE+
int_list     ::= INT (',' INT)*
```

A.2 Constraint Rules

1. Phase marker is the first non-whitespace character of every non-blank line
2. Phase marker must be followed by a single space
3. `from=N` requires $N <$ current line number (no forward references)
4. Maximum indent depth: 2 levels (4 spaces)
5. `?tag` slots appear only in `:` lines
6. `conf` must be in $[0.0, 1.0]$
7. In `conf_range=L..H`, $L \leq H$

B Dataset Schema

```
{
  "id":                 string,    // e.g. "v1_042_0"
  "domain":             string,    // medical | code | math | ...
  "difficulty":         string,    // simple | medium | complex
  "void_depth":         string,    // single | cascade
  "task":               string,    // task description
  "void_instruction":   string,    // void-B specific instruction
  "wcy_reasoning":      string,    // full WCY trace
  "quality": {
    "parse_rate":        float,    // fraction of lines parseable
    "void_generated":    int,      // number of ?tags in trace
    "void_resolved":     int,      // number of ?tags resolved
    "resolution_rate":   float,    // void_resolved / void_generated
    "infer_count":       int,
    "act_count":         int,
```

```
    "warning_count":    int,
    "usable":           bool      // passed all quality gates
  },
  "model":              string,
  "spec_version":       string,    // "1.1"
  "type":               string,    // void_resolution_cycle | void_cascade
  "usable":             bool
}
```